

07：阶段 1——真实收益标签

1. 什么是训练标签

监督学习需要一组“输入—答案”样本。模型读取输入，产生预测，再用预测与答案之间的差异调整参数。这里的“答案”叫作标签。

第二版未来的一条区域训练样本可以粗略写成：

输入：历史窗口 H 的道路节点需求表示 + 候选区域 R
答案：区域 R 在后续窗口 Y 上的真实平均查询工作量收益 y_R

当前标签、统一模型输入和无传播 CUDA MLP 都已完成；下一步是 NBFNet 主模型。

2. 主标签的精确定义

代码当前采用：

单条查询收益
= 原图双向 Dijkstra 展开节点数
- 单区域压缩查询总展开节点数

其中：

单区域压缩查询总展开
= 压缩图搜索展开
+ 端点局部接入展开

一个区域的标签是所有选定标签查询的单条收益平均值：

y_R = 所有标签查询收益之和 / 标签查询数量

源码常量 `LABEL_WORK_DEFINITION` 直接记录了这个口径，位于 `src/region_labels.py`。

3. 一个具体数字例子

假设标签窗口只抽三条查询：

查 询	原图展 开	压缩图搜索展 开	局部接入展 开	总压缩展 开	收 益
q1	1,000	760	40	800	200
q2	500	510	10	520	-20
q3	900	600	0	600	300

总收益是 $200 - 20 + 300 = 480$ ，平均收益标签是 $480 / 3 = 160$ 。

负的单条收益是允许的，表示这条查询上局部接入或图结构导致总展开反而增加。模型学习的是整个查询负载上的平均价值，不要求每条查询都受益。

4. 为什么每次只构建一个区域

对候选 R 打标签时，代码调用：

```
build_compression_index(graph, [R])
```

方括号中只有一个区域。这样测到的是“R 单独存在时”的边际收益，不混入其他候选贡献。

这也解释了为什么候选池里区域可以互相重叠：它们不会在同一次标签查询中同时物化。

5. 每个候选的标签生成步骤

对区域 R：

1. 构建只包含 R 的精确压缩索引；
2. 对固定的每一条标签 OD，取得原图基线结果；
3. 关闭端点缓存，运行单区域压缩查询；
4. 比较原图距离与压缩距离；
5. 记录压缩图展开、局部接入展开和总展开；
6. 计算单条查询工作量差；
7. 聚合平均收益、正收益查询比例、端点接入率等字段；
8. 若任意距离不一致，正确率就会低于 100%，候选不能通过阶段门。

6. 基线为什么可以复用

同一批标签查询在原图上的结果与候选无关，所以无需为 1,200 个候选重复计算 1,200 次。

脚本优先从全量基线明细加载每个 `query_id` 的：

- 原图距离；
- 原图展开节点数；
- 当时记录的毫秒耗时。

如果文件不存在，或显式使用 `--recompute-baseline`，才重新计算选定查询的基线。

7. 时间窗口和正式抽样

完整标签窗口 Y 是全部 OD 时间比例 `[0.35, 0.70)`，约 34,329 条。

正式第一轮没有立刻让 1,200 个候选都跑完整 Y，而是固定：

```
1,200 个候选
× Y 中随机种子 42 抽取的 2,000 条查询
= 2,400,000 次单区域压缩查询
```

2,000 条是当前冻结的正式抽样预算：它足以完成全候选监督和模型可学习性验证，同时把完整

Y 窗口保留为模型结构冻结后的高成本复核。不能把旧候选池上的独立样本筛查当成当前全图随机池的稳定性证明；当前结论只对应这批固定的 2,000 条查询。

8. 为什么抽样后还要固定 `query_id`

“随机抽 2,000 条”如果每次运行都重新抽，会改变标签定义。脚本因此把实际抽中的全部 `query_id` 写入标签清单。

以后断点续跑时，不只是随机种子要相同，实际查询编号列表也必须完全一致。

9. 多进程与断点续跑

1,200 个候选互相独立，所以脚本使用多个进程并行评测不同候选。每个工作进程持有：

- 相同道路图；
- 相同 2,000 条查询；
- 相同基线结果；
- 当前被分配的候选区域。

每完成一个候选，主进程立刻：

1. 向 CSV 追加一行；
2. 刷新文件；
3. 更新清单中的已完成区域 ID；
4. 把状态写成 `in_progress` 或 `complete`。

中断后重新运行时，脚本读取已有 CSV，只提交尚未完成的区域。

10. 为什么续跑前严格比较运行身份

以下任一项与已有清单不一致时，脚本拒绝继续追加：

- 候选摘要；
- 目标区域 ID；
- 标签窗口比例；
- 实际 query_id 列表；
- 工作量定义；
- 端点缓存容量；
- 候选抽样种子；
- 查询抽样种子。

否则一个 CSV 可能混合两种实验条件，后续分析无法解释。

`--restart` 会删除原输出重新开始，只应在明确知道要抛弃已有进度时使用。

11. CSV 每个字段是什么意思

正式输出位于

`results/gnn_v2/region_training_labels.csv`。

主要字段分为几组。

区域身份

- `region_id`: 候选编号；
- `selection_method`: 候选生成方法，当前统一为 `fixed_random_bfs`；
- `seed_node`: BFS 种子；
- `node_count`: 区域节点数；
- `boundary_count`: 边界节点数；
- `internal_node_count`: 压缩后删除的内部节点数；
- `shortcut_count`: 单区域生成的 shortcut 数。

查询覆盖

- `label_query_count`: 用于打标的查询数, 正式结果为 2,000;
- `reachable_query_count`: 原图上可达的查询数;
- `endpoint_access_query_count`: 至少一个端点在区域内部的查询数;
- `endpoint_access_rate_pct`: 上述比例。

工作量

- `baseline_avg_expanded`: 原图平均展开;
- `indexed_avg_expanded`: 压缩查询总平均展开;
- `indexed_avg_graph_expanded`: 压缩图搜索平均展开;
- `indexed_avg_access_expanded`: 局部接入平均展开;
- `avg_workload_gain`: 主标签;
- `total_workload_gain`: 全部查询收益之和;
- `positive_gain_query_rate_pct`: 收益大于 0 的查询比例。

辅助时间和正确性

- `baseline_avg_ms_unpaired`: 保存的基线平均毫秒;
- `indexed_avg_ms_unpaired`: 当前压缩运行平均毫秒;
- `indexed_p95_ms_unpaired`: 当前压缩运行 P95;
- `preprocessing_seconds`: 单区域索引构建时间;
- `evaluation_seconds`: 该区域所有查询评测时间;
- `correctness_rate`: 距离正确率。

字段名中的 `unpaired` 提醒读者: 基线和压缩耗时不是在同一进程、紧邻交替执行, 不应拿它们直接声明在线加速。

12. 当前正式标签的实际状态

标签清单

results/gnn_v2/label_manifest.json 当前显示：

- completed_region_count = 1200;
- target_region_count = 1200;
- status = complete;
- CSV 一共 1,201 行，即 1 行表头加 1,200 个候选；
- 所有行 label_query_count = 2000;
- 最低 correctness_rate = 1.0。

这意味着正式 1,200 × 2,000 标签计算已经完成。

13. 正式标签的分布

根据当前正式 CSV 重新分析：

指标	值
最小平均收益	0.200
P10	4.298
中位数	25.899
P90	104.05
最大平均收益	5
	183.41
	4
全部候选均值	39.695

指标	值
标准差	40.853
平均收益为正的候选比例	100%

标准差明显大于零，说明标签不是几乎相同的常数；候选之间存在可供模型学习的稳定价值差异。

14. 当前统一事实口径

正式标签 CSV、标签清单、正式分析 JSON/Markdown 和主线文档现在采用同一口径：1,200 个候选均已完成 2,000 条固定查询标签，清单状态为 `complete`，分析阶段状态为 `ready_for_modeling`。这表示正式抽样标签阶段门已经通过，可以进入模型训练；完整 Y 窗口仍保留为模型取得正向结果后的复核，不属于当前阶段的未完成任务。

15. 还不能据此宣布第二版成功

标签完成只说明监督数据准备好，不说明 NBFNet 传播有效。接下来至少还要：

1. 已完成的无传播区域 MLP 确认标签可以被历史特征稳定学习；
2. 实现并训练 OD 条件化双向 NBFNet；
3. 与 Proxy、固定扩散、MLP、无向传播等严格比较；
4. 在固定预算区域集合和未来隔离窗口上做精确在线评测。

下一章专门解释标签分析中的 Oracle。